Interface-Scoped Dispatch: Independent Components with O(1) Self-Calls

R. SCOTT MCKINNEY, Manifold Systems, USA

Object composition under static typing forces a trade: independent components, or open recursion. Forwarding-based constructs compose independent components, but a component's self-calls do not dispatch through the composite. Trait, mixin, and family-based composition preserves self-call dispatch but fuses the components into the class or binds them to an enclosing family, giving up their independence.

We present *Parts*, a practical model for object composition with independent components and open recursion, realized as a compiler plugin for Java 8–26. A *part* is an independent component, supplied as a runtime object at construction, whose self-calls route through the composite; the enabling mechanism, *interface-scoped dispatch* (ISD), assigns composite identity per interface rather than per object. Self-calls route through the correct composite in arbitrary compositions. The dispatch cost is O(1), empirically equivalent to a vtable call.

1 INTRODUCTION

Object composition is the standard prescription for avoiding implementation inheritance [1]. Two properties make it a genuine replacement: *independent components* (late-bound objects in arbitrary compositions) and *open recursion* (self-calls through the composite). Static typing is what makes the two hard to combine (Section 7).

Forwarding-based composition (Kotlin `by` [25], Lombok `@Delegate` [26]) supplies independent components but does not support open recursion: a component's self-calls resolve within the component because the composite is never on the dispatch path. This is the *self* problem [3, 8].

Trait composition (Scala `with` [10], mixins [4]) inverts the trade. Self-calls dispatch through `this` in the linearized class, so composite overrides are visible. But traits are fused into the class hierarchy at compile time, not supplied as late-bound objects. Components cannot be constructed independently or supplied externally; composition is static.

Parts provides both. `@part` marks a class as a *part*, an independent object supplied at construction. `@link` marks a field through which a composite delegates one or more interfaces to a part. When a composite is constructed, a compiler plugin wires composite identity into each part, so self-calls in the part dispatch through the composite. A link field accepts all implementations of the field's declared type: forwarding works with any object; part objects bring open recursion. Parts thus subsumes forwarding rather than replacing it. The enabling mechanism is *interface-scoped dispatch* (ISD): assigning composite identity per interface rather than per object enables open recursion in all Parts compositions.

The core contributions are:

- **Parts**: a model for object composition in which components are independent runtime objects, self-calls dispatch through the composite, and interface contracts are statically enforced. The model is defined over standard interface hierarchies with a central dispatch invariant (Invariant 1) and composite-scoped encapsulation (`@internal`).
- **Interface-scoped dispatch (ISD)**: the mechanism that assigns composite identity per interface rather than per object, enabling open recursion in Parts compositions, including partial delegation (where a composite claims a proper subset of a part's interfaces) and arbitrary compositions.
- **An implementation**: a compiler plugin for Java 8–26 realizing the model at O(1) dispatch cost, with open-source release [29] and IDE integration.

Author's address: R. Scott McKinney, Manifold Systems, USA, scott@manifold.systems.

2 OVERVIEW

Consider a minimal example that shows `@part` and `@link` working in a concrete single-rooted composite.

```
interface Actor {
    void attack();
    void takeAction();
}

@part class Hero implements Actor {
    public void attack() { out.println("Swing club!"); }
    public void takeAction() { attack(); }
}

class Wizard implements Actor {
    @link Actor actor;
    Wizard(Actor actor) { this.actor = actor; }
    @Override public void attack() { out.println("Cast spell!"); }
}

new Wizard(new Hero()).takeAction(); // "Cast spell!"
```

When `Wizard` is constructed, the compiler wires `Hero`'s composite identity for `Actor` to `Wizard`. A call to `takeAction()` dispatches to `Hero.takeAction()`, and the self-call `attack()` within it dispatches back through `Wizard`, invoking `Wizard.attack()`, not `Hero.attack()`.

This is the decorator pattern with one addition: the wrapped object's self-call routes back through the composite. A plain decorator would forward `takeAction()` to the `Hero`, whose `attack()` call would stay on `Hero` ("Swing club!"), losing the override.

Because the link field accepts any `Actor` at construction time, the part is a real late-bound object, not fused into the class hierarchy. A different `Actor` implementation can be substituted without changing `Wizard`.

Interface contracts are enforced at compile time: the link field has type `Actor`, and the compiler verifies that `Wizard` implements the delegated interface. Part identity may not escape the composite dispatch path: passing or assigning `this` as the concrete part type is a compile error; interface types are required.

This example is single-rooted: `Wizard` is the sole composite for `Hero`, and one composite identity suffices. The following section exposes the limit of the single-identity model and the per-interface generalization that resolves it.

3 THE MODEL

Motivating example

The following example shows a part whose interfaces are rooted at different composites. `Hero` implements both `Combatant` and `Actor`, and two wrappers decorate it: a `Wizard` changes how it fights (`Combatant`), while a `Demon` partially possesses it, seizing its name and will (`Actor`) but leaving its trained combat untouched. As in the overview, the wrapped hero's self-calls route back through the wrappers; now two wrappers claim different interfaces of the same part. So from a single `Hero` instance, `attack()` must route through `Wizard` and `name()` through `Demon`; a single composite identity cannot serve both. Assigning a separate composite identity per interface resolves this. Figures 1 and 2 show the structure.

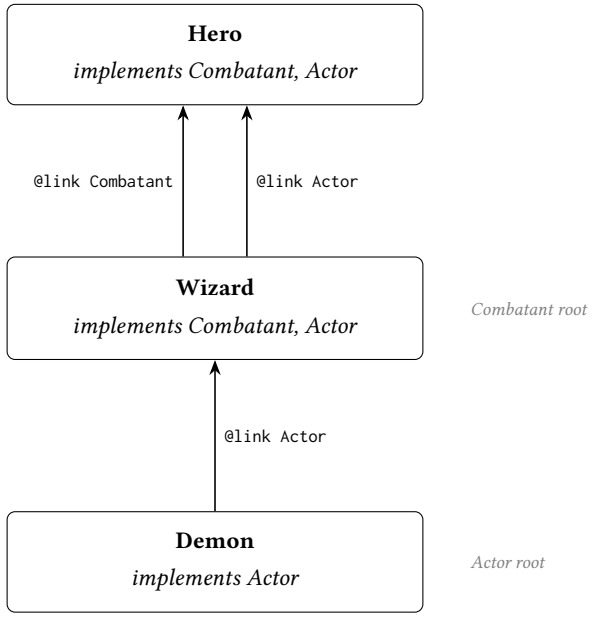

Fig. 1. Composition structure. Arrows show link fields labeled by the delegated interface. `Wizard` is the root for `Combatant` dispatch; `Demon` is the root for `Actor` dispatch.

```
interface Combatant {
  void attack();
}

interface Actor {
  String name();
  void takeAction();
}

@part class Hero implements Combatant, Actor {
  public void attack() { out.println("Swing club!"); }
  public String name() { return "Milo"; }
  public void takeAction() {
    out.print(name() + " acts: ");  // Actor dispatch
    attack();                       // Combatant dispatch
  }
}

@part class Wizard implements Combatant, Actor {
  @link Combatant combatant;
  @link Actor actor;
  Wizard(Hero h) { this.combatant = h; this.actor = h; }
  public void attack() { out.println("Cast spell!"); }
}

// A demon partially possesses the hero: it seizes the Actor facet
// (name and will) but not the Combatant facet (trained combat).
class Demon implements Actor {
  @link Actor actor;
  Demon(Actor a) { this.actor = a; }
  public String name() { return "Diablo"; }
}

Wizard wizard = new Wizard(new Hero());
Demon demon = new Demon(wizard);
demon.takeAction();
// Actor dispatch rooted at Demon:       name()   -> "Diablo"
// Combatant dispatch rooted at Wizard:  attack() -> "Cast spell!"
// prints:  Diablo acts: Cast spell!
```

Fig. 2. Composition with multiple composite roots per part. `Hero` carries distinct composite identities for `Combatant` (rooted at `Wizard`) and `Actor` (rooted at `Demon`).

Definitions

We define Parts over a standard typed object model where classes implement interfaces and method dispatch is defined by the receiver's runtime type.

A *part* is a class annotated with `@part`; parts implement one or more interfaces, and a part may itself be a composite. A *link* is a field annotated with `@link`, whose declared type is an interface or an abstract part class; a link claims the interfaces common to its field type's hierarchy and the enclosing composite's, and the plugin enforces `private` and `final` on it (explicit modifiers are

redundant). A *composite* is a class containing one or more link fields, and it implements every interface its links claim. The *delegation surface* of a part is its interface hierarchy, excluding any interface marked `@internal` in its `implements` clause.

When a part is wired to a composite it receives a *composite identity*: a reference to the composite object, through which self-calls within the part—calls to methods on its delegation surface—dispatch rather than through `this`. The enabling mechanism is *interface-scoped dispatch* (ISD): the receiver of a self-call is determined by the interface of the called method, not by `this`, so each interface in a part's delegation surface carries an independent composite identity and self-calls route through the composite for that interface.

A part may itself contain link fields, in which case composite identity propagates transitively. When such a part is wired to an outer composite, that composite becomes the dispatch root for each interface it claims throughout the delegation chain, while interfaces it does not claim retain the roots established by inner composites. This *transitive propagation* traverses part links only: it terminates at any non-part link, a forwarding leaf whose self-calls remain local to the delegate. A composition may thus freely mix parts and non-part delegates, each part routing and each non-part forwarding, with no interaction between them.

Composites must also resolve *diamonds*: when two links share a common interface, exactly one must declare `@link(share=T.class)` for it. That link is authoritative; the other delegates only its non-shared interfaces, and the shared interface is excluded from propagation when the non-authoritative link is wired. Unresolved diamonds are compile errors.

Two practical extensions, composite-scoped encapsulation (`@internal`) and abstract parts with deferred linkage (`asLink()`), are defined and implemented (Section 4).

Dispatch rule. Traditional object-oriented dispatch resolves a call through the receiver's object identity:

$$\text{call } m() \text{ on } o \;\longrightarrow\; \mathit{vtable}(o,\, m)$$

Interface-scoped dispatch replaces the single object identity with a per-interface composite identity. A self-call $m()$ in part P where $m \in I$ resolves through the composite identity for I:

$$\text{self-call } m() \in I \text{ in } P \;\longrightarrow\; \$\mathit{selves}_P[I].m()$$

For self-calls within a part's methods, the dispatch triple is *(part, interface, method)* rather than the conventional *(object, method)*. Figure 3 contrasts this per-interface routing with a single shared composite identity. This is *open recursion*: a method's self-calls late-bind to the most-derived receiver. Here the receiver resolves from the interface of the self-call. Forwarding is the closed case: a component's self-calls stay in the component, and the composite never enters the recursion.

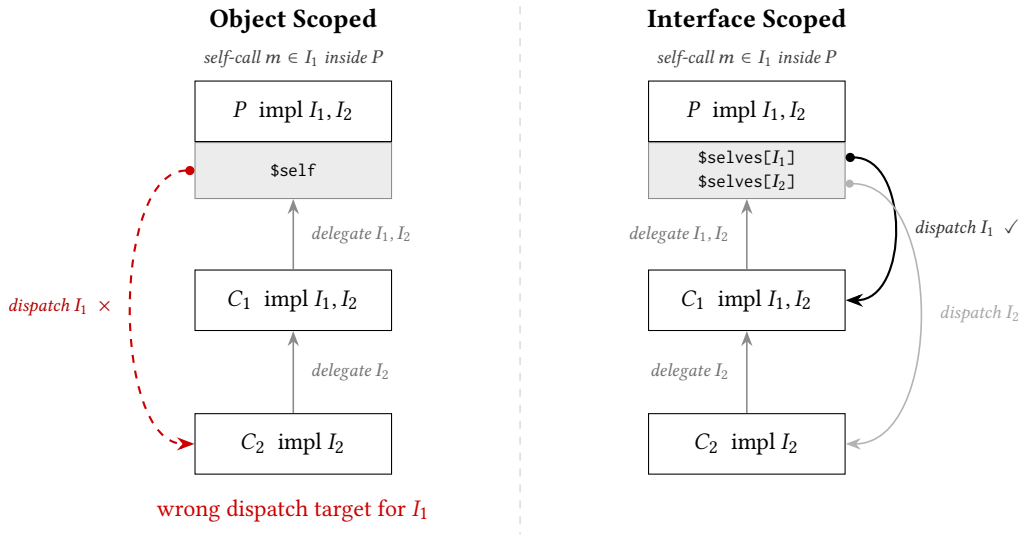

Fig. 3. Object-scoped vs. interface-scoped dispatch. In both panels C_2 (impl I_2) delegates I_2 to C_1 (impl I_1, I_2), which delegates both interfaces to part P. *Left:* a self-call $m() \in I_1$ inside P routes through `$self` = C_2, the wrong composite for I_1. *Right:* `$selves[`I_1`]` routes to C_1; `$selves[`I_2`]` routes to C_2; each self-call reaches its correct composite root.

INVARIANT 1. *For any part P in a delegation chain for interface I rooted at composite C: a self-call `this.m()` within P where $m \in I$ dispatches to C.*

This invariant presumes each interface has a single composite root. The model maintains this precondition for every composition it admits but does not statically enforce it; the lone exception is treated in Section 5.4.

The invariant is established by construction. When composite C establishes a link to part P for interface I, the plugin-generated `$linkPartToSelf` sets $\$selves_P[I] \leftarrow C$ and recurses into P's own links, propagating C's identity for each interface C claims and leaving all other slots unchanged. Each interface of a part resolves through a single `$selves` slot. Wiring an outer composite that claims I re-roots this slot to that composite and leaves the slots for interfaces it does not claim unchanged, so the slot holds exactly one root at any time and stabilizes once the outermost composite claiming I is wired; C denotes that final root. Each write occurs during the wiring composite's construction. Wiring happens solely as link field assignment, and the plugin enforces `final` on link fields, so writes are confined to construction: the `$selves` slots are never written after the outermost composite claiming I is constructed.

Visibility follows from safe publication: once the composite root is safely published, the Java Memory Model guarantees that any thread observing it sees all writes that happened-before publication, the wired slots included. The role of `final` here is write-confinement, not the JMM's final-field freeze: array-element writes are not covered by freeze semantics, so the guarantee rests on write-confinement together with safe publication of the root, not on the finality of the slots.

Self-calls in P where $m \in I$ are rewritten by the plugin to `((I) $selves[IDX_I]).m()`; when m belongs to multiple interfaces (the diamond case), I is determined at compile time by diamond resolution (Section 4). For self-calls through the delegation surface, there is no code path in a part method that dispatches through anything other than the indexed slot. Because the slot holds C

once the composition is constructed and is not written thereafter, the invariant holds at every point a self-call can execute. Before wiring, each slot is initialized to `this`; an unwired part dispatches on itself, consistent with standard object-oriented dispatch. In this initial state, the invariant holds trivially. A full mechanized proof in a Featherweight Java subset is a natural direction for follow-on work (Section 6.4).

4 IMPLEMENTATION

The implementation is a `javac` plugin operating on the compiler's AST during two phases. In ENTER, the plugin generates structural scaffolding: the `$selves` composite identity array and `$linkPartToSelf` wiring method for each part, delegation forwarding method for each composite class, and (for abstract parts) the `$Impl` subclass and `asLink()` factories. In ANALYZE, after type resolution, self-calls in part methods are rewritten, link field assignments are rewritten to invoke wiring, and `@internal` visibility is enforced. No bytecode manipulation is performed; all transformation is at the AST level and subject to standard type checking.

4.1 Part Transformation

Each part holds a `$selves` array whose compile-time-determined ordinals cover the part's interfaces.

```
private final Object[] $selves = {this, ..., this};
// after wiring: $selves[IDX_I] = composite identity for interface I
```

Before wiring, every `$selves` slot holds `this`, so the part dispatches on itself. Wired at link assignment via `$linkPartToSelf`, dispatch routes through the composite defining the link. `$linkPartToSelf` updates each relevant slot and recurses through link fields in the part whose delegation surface includes the interface being propagated, with cycle detection at each assignment.

Within a part method, every call `this.m(args)` where `m` is declared in interface I is rewritten to:

```
((I) $selves[IDX_I]).m(args)
```

The cast to I selects the dispatch interface explicitly; when diamond resolution has assigned m to a specific authoritative link, I is that link's interface, determined at compile time.

The ordinal `IDX_I` is assigned per part over its own delegation surface, so it is local to that class: the rewritten self-calls and the generated `$linkPartToSelf` both reside in the part and share its indexing, and no index agreement across separately-compiled classes is required.

4.2 Composite Transformation

A composite implements every interface claimed by its links. For each method of a claimed interface that the composite does not implement directly, the plugin generates a delegating method that forwards through the corresponding link field:

```
R m(args) { return link.m(args); }
```

When two links share an interface, the forwarders for the shared methods route through the authoritative link (Section 3). The non-authoritative link's part need not hold the authoritative instance: its `$selves` slot for the shared interface points at the composite, so the shared methods dispatch through the composite to the authoritative link. Where a part implements the shared interface directly, that implementation is overridden by delegation, with no instance shared.

In composite classes, each link field assignment `this.link = expr` is rewritten to invoke `$linkPartToSelf`, propagating the composite identity into the part. An `instanceof` check guards the call so that non-part assignments pass through unchanged.

4.3 Abstract Part Support

A part may be abstract, deferring some interface methods to the composite. Linking an abstract part obligates the composite to implement those methods or to be declared abstract, enforced at compile time: the plugin generates a delegating method for every interface method the part implements and none for a method it leaves abstract, so a concrete composite is missing exactly those methods and `javac` rejects it unless they are supplied directly. This is the compositional analog of the rule that a concrete class must implement its inherited abstract methods.

To make an abstract part instantiable for linking, the compiler generates a private `$Impl` subclass whose stubs throw `DelegationLinkageError`, and an `asLink()` factory per non-private constructor:

```
private static final class $Impl extends Foo {
    $Impl(/* ctor args */) { super(/* ctor args */); }
    @Override public T m(Params p) {
        throw new DelegationLinkageError(
            "abstract method called on an unlinked part");
    }
}
public static Foo asLink(/* ctor args */) { return new $Impl(/* same */); }
```

Once wired, the stubs are unreachable: self-calls dispatch through `$selves` to the composite, whose implementation the compile-time obligation guarantees. The stubs remain only as a runtime guard for an `asLink()` instance invoked before it is wired.

4.4 Default Methods

A part inherits self-calling behavior from the default methods of the interfaces it implements. If neither the composite nor the part overrides a default, it runs with the part as receiver, not the composite, resulting in incorrect self-call dispatch.

No bytecode instruction directly invokes a default with a receiver other than the current `this`, but a method handle bound to an `invokedynamic` call site can. When a part does not override a default, the plugin generates an override that invokes the interface's default implementation through such a site, with the receiver taken from the interface's slot in `$selves`, so the default's self-calls dispatch through that receiver. An explicit `I.super.m()` in part code is rewritten the same way.

```
interface Foo {
    void bar();
    // default self-calls bar()
    default void foo(P p) { bar(); }
}

@part class FooPart implements Foo {
    public void bar() { ... }

    // generated
    // calls Foo.super.foo() with composite in $selves as receiver
    // Foo's bar() self-call dispatches through the composite
```

```
    public void foo(P p) {
        invokedynamic Foo::foo((Foo) $selves[IDX_Foo], p);
    }
}
```

The site's bootstrap links it once to a fixed target (the default implementation), with the receiver as its only varying argument. Being monomorphic, it is inlined by the JIT to the same type of array-indexed invocation as in Section 5.

4.5 Safety Guarantees

Cycle detection. If a part is wired to itself directly or transitively, `DelegationLinkageError` is thrown at assignment time during the recursive `$linkPartToSelf()` call.

Deferred method access. The obligation that a wired composite implement an abstract part's deferred methods is enforced at compile time; an `asLink()` instance invoked before wiring throws `DelegationLinkageError` from the `$Impl` stub, the compositional analog of `AbstractMethodError`.

`@internal` *enforcement.* `@internal` may annotate an interface method or an interface type literal in a class's `implements` clause, with distinct effects. A marked method has its calls confined to the composite, the intent behind `protected` in implementation inheritance, but more cleanly scoped for composition. A marked interface type literal in a class's `implements` clause removes that interface from the class's delegation surface: no composite may claim it via a link field.

Self-preservation. Inside a part, `this` use is guarded. Field access, private calls, and callbacks through the part's own link fields (the compositional analog of `super`) are all permitted. Passing or assigning `this` as the concrete part type is not: that is a compile error, since a caller holding the bare part could invoke methods outside the composite dispatch path and violate Invariant 1. The only `this` that can leave is therefore either interface-typed or `Object`-typed: resolving to the composite claiming the interface or the part itself. A client receiving it reaches the composite's overrides.

5 EVALUATION

5.1 Dispatch Performance

The dispatch path for a self-call through a composite is:

(1) Load `$selves[IDX_I]`, an array-indexed load with compile-time constant index
(2) Cast to I
(3) `invokeinterface m`, standard JVM virtual dispatch

This is O(1). The parallel to vtable dispatch is direct: a vtable call is an indexed slot load followed by an indirect call. Call sites in typical compositions are monomorphic or bimorphic, the JIT's most aggressively optimized scenarios. Composite dispatch adds at most one additional virtual dispatch over direct delegation.

Benchmark. We verify this empirically using JMH [28] in average-time mode. Each scenario calls `compute()` through a composite root at chain depth d (1–6). The *baseline* traverses d `@link` delegation hops to a leaf that returns directly, with no self-call. The *concrete* variant traverses the same hops but the leaf issues a self-call (`$selves[IDX_I].scale()`) from a concrete part method, dispatching through the composite root. The *default* variant issues the same self-call from an inherited interface default method the leaf does not override, routed through the generated `invokedynamic` override of Section 4.4. The delta (concrete minus baseline) isolates the cost of the

self-call dispatch mechanism alone. All runs: 10 JVM forks, 5 warmup iterations, 10 measurement iterations per fork (100 data points per cell); errors are ± half-width of the 99.9% confidence interval. A separate single-dispatch calibration run (`invokeinterface` on a monomorphic call site, same harness) yields 0.43 ns/op; the delta values in Table 1 are therefore roughly 1–2× a single virtual dispatch.

Choice of baseline. The delta is measured against Parts's own forwarding path, which is the right baseline: that path is semantically equivalent to Kotlin by and Lombok `@Delegate` (Section 7), so the delta prices exactly the self-call routing those mechanisms cannot express, against their own dispatch cost. A cross-system benchmark against other composition models would price differences in implementation strategy rather than the self-call dispatch this delta isolates, so that comparison is analytic (Section 7).

Table 1. Self-call dispatch cost by delegation depth (JMH, average-time mode; ns/op, ± 99.9% CI half-width). *Baseline* traverses the delegation hops with no self-call; *Concrete* and *Default* add one self-call through the composite, issued from a concrete part method and from an inherited interface default (via `invokedynamic`) respectively. The delta (Concrete minus Baseline) isolates the self-call dispatch cost over plain delegation. Values are rounded from the measured means and intervals; the delta combines the Baseline and Concrete cells (difference of means, half-widths in quadrature) and may differ from the rounded column difference by 0.01 ns.

Depth	Baseline (ns)	Concrete (ns)	Default (ns)	Delta (ns)
1	0.62 ± 0.02	1.16 ± 0.03	1.18 ± 0.03	0.55 ± 0.03
2	0.91 ± 0.03	1.56 ± 0.04	1.55 ± 0.04	0.65 ± 0.05
3	1.18 ± 0.03	1.93 ± 0.05	1.92 ± 0.05	0.75 ± 0.06
4	1.64 ± 0.03	2.30 ± 0.06	2.23 ± 0.06	0.66 ± 0.07
5	1.93 ± 0.06	2.70 ± 0.06	2.76 ± 0.07	0.77 ± 0.09
6	2.37 ± 0.06	3.21 ± 0.09	3.23 ± 0.08	0.84 ± 0.11

Across depths 1–6 the delta remains bounded, on the order of a single virtual dispatch (the 0.43 ns calibration above), consistent with one array-indexed load (`$selves[IDX_I]`) followed by `invokeinterface`. The self-call routes through a dynamically wired `$selves` slot, so its receiver is not statically known and the dispatch does not fold away: the delta is one genuine virtual dispatch, with the array indexing on top (confirmed free below). The O(1) claim is structural: one dispatch per self-call, independent of depth, with the per-call cost pinned by the isolation measurement below. The small upward drift in the delta across depths (0.55 to 0.84 ns) stays well under a second dispatch and plausibly reflects less complete inlining of the dispatch site within the larger compiled method at greater depth. The *default* column tracks the *concrete* column within ±3% at every depth, with the sign of the difference varying; the two are indistinguishable at this resolution. The interface-default template, dispatched through `invokedynamic`, thus carries no measurable cost over the concrete part template, confirming the equivalence claimed in Section 4.4. The comparison is steady-state: the one-time bootstrap that links each `invokedynamic` site is absorbed during warmup and is not measured here.

Isolating the array index. The depth-series delta is essentially one virtual dispatch; to confirm the `$selves` array indexing adds nothing on top, we isolate the dispatch. Holding a target method non-inlinable so a real call occurs, we compare three depth-0 forms over the same receiver: a vtable call (`invokevirtual`), the `invokeinterface` it compiles to, and the plugin's emitted form (`((I) $selves[IDX_I]).m()` (an `Object[]` element load, a cast, and `invokeinterface`). Over 10

JVM forks (JDK 21) the three are statistically indistinguishable: the ISD form differs from the bare `invokeinterface` it emits by 0.03 ± 0.09 ns, and from `invokevirtual` by 0.10 ± 0.05 ns, both within the fork-to-fork variance of the measurement. The array-element load is thus not separately observable; it overlaps with the dispatch. The self-call costs one virtual dispatch and no more, the empirical content of the vtable-equivalence claim.

Construction cost. The result above is the steady-state call cost; wiring is paid once, at construction. Building a composite invokes `$linkPartToSelf` at each link assignment, which sets the relevant `$selves` slots and recurses through the part's own link fields, propagating composite identity for each interface the composite claims. Cycle detection adds only a constant-time check per claimed interface, an `aaload` and a compare, so each part node costs O(1) per propagated interface and the whole traversal is linear in the size of the delegation graph reachable through link fields. None of it lies on the dispatch path, so the per-call cost is unchanged, and the traversal is proportional to the object graph the program constructs in any case. The cost is therefore a bounded, one-time increment to composite construction, amortized over every subsequent self-call.

5.2 Example: Teaching Assistant

The following example exercises diamond dispatch resolution and self-call routing through the composite. A teaching assistant is both a student and a teacher; both roles share a common `Person` base.

```
interface Person {
  String name();
  String title();
  String titledName();
}
interface Student extends Person { ... }
interface Teacher extends Person { ... }
interface TA extends Student, Teacher { ... }

@part abstract class PersonPart implements Person {
  private final String name;
  public PersonPart(String name) { this.name = name; }
  public String name() { return name; }
  public String titledName() { return title() + " " + name(); }
}

@part class StudentPart implements Student {
  @link Person person;
  public StudentPart(Person p) { this.person = p; }
  public String title() { return "Student"; }
}

@part class TeacherPart implements Teacher {
  @link Person person;
  public TeacherPart(Person p) { this.person = p; }
  public String title() { return "Teacher"; }
}

@part class TaPart implements TA {
  @link(share=Person.class) Student student;
```

```
  @link Teacher teacher;
  public TaPart(Student student) {
    this.student = student;
    this.teacher = new TeacherPart(student);
  }
  public String title() { return "TA"; }
}

TA ta = new TaPart(new StudentPart(PersonPart.asLink("Milton")));
ta.titledName(); // "TA Milton"
```

Under ordinary object composition (forwarding), the shared `Person` methods must each be resolved by hand, and `titledName()` still calls `title()` on the delegate's own `this`. Parts resolves both problems.

`PersonPart` holds the template once. Its self-call to `title()` dispatches through the composite at each level of the delegation chain.

`titledName()` is defined once in `PersonPart` and overridden nowhere. Diamond resolution is handled statically: `@link(share=Person.class)` designates `student` as the authoritative delegate for `Person`, `TaPart` transitively claims `Person` throughout the composite.

5.3 Case Study: Stream Decoration

A pattern common in the JDK, exemplified by `java.io.InputStream` [1], is a bulk operation implemented as a loop over a finer one: `read(byte[], int, int)` calls the single-byte `read()` repeatedly. Decorators override `read()` to add behavior; the template is supposed to propagate it to bulk calls automatically. The `Readable` interface below captures this contract directly. Under forwarding, the propagation breaks.

A `CountingStream` that tracks bytes consumed overrides `read()` to increment a counter. With Lombok's `@Delegate`, `read(byte[], int, int)` is forwarded to the delegate, which calls `read()` on itself, not on `CountingStream`. The override is never reached for bulk calls.

Forwarding [26].

```
interface Readable {
    int read();
    int read(byte[] buf, int off, int len);
}

class BaseStream implements Readable {
    private final byte[] data;
    private int pos;
    BaseStream(byte[] data) { this.data = data; }
    public int read() {
        return pos < data.length ? (data[pos++] & 0xff) : -1;
    }
    public int read(byte[] buf, int off, int len) {
        for (int i = 0; i < len; i++) {
            int b = read();                      // self-call
            if (b == -1) return i == 0 ? -1 : i;
            buf[off + i] = (byte) b;
        }
        return len;
```

```
    }
}

class CountingStream implements Readable {
    @Delegate private final Readable delegate;
    private int count;
    CountingStream(Readable r) { this.delegate = r; }
    @Override public int read() {
        int b = delegate.read();
        if (b != -1) count++;
        return b;
    }
    public int count() { return count; }
}

CountingStream cs = new CountingStream(new BaseStream(data));
cs.read(buf, 0, 16);  // -> delegate.read(byte[],int,int)
                      //    -> delegate.read()  -- not cs.read()
cs.count();           // 0  -- bulk reads not counted
```

Parts. `BaseStream` is unchanged except for the `@part` annotation. The self-call in `read(byte[], int, int)` routes through `CountingStream` on every call path.

```
@part class BaseStream implements Readable { /* same as above */ }

class CountingStream implements Readable {
    @link Readable stream;
    private int count;
    CountingStream(Readable r) { this.stream = r; }
    @Override public int read() {
        int b = stream.read();
        if (b != -1) count++;
        return b;
    }
    public int count() { return count; }
}

CountingStream cs = new CountingStream(new BaseStream(data));
cs.read(buf, 0, 16);  // -> BaseStream.read(byte[],int,int)
                      //    -> self-call read() -> CountingStream.read()
cs.count();           // 16  -- all bytes counted
```

A new decorator (buffering, decryption, rate limiting) requires only overriding `read()`; `BaseStream.read(byte[], int, int)` routes its self-call through whichever composite is the dispatch root, without modification.

5.4 Limitations

- Using `@link`, `@part`, and `@internal` requires configuring the `javac` plugin in the build system (Maven, Gradle, or similar).
- A part instance carries one composite root per interface (Section 6). Linking the same instance into two composites that each claim the *same* interface, with neither nesting the

other, re-roots that interface to whichever is wired last, redirecting the first composite's self-calls. The contention is undetectable at the wiring step: re-rooting an already-wired slot is the model's normal operation during transitive propagation, so the two cases are observationally identical. One instance cannot be the composite root for a single interface in two places at once; separate part instances avoid it entirely.
- The plugin relies on internal `javac` APIs for AST rewriting, which may require adaptation across JDK releases. In practice the approach has proven stable: the Manifold project, on which this plugin is built, has tracked 18 JDK releases without API-driven breakage.

6 DISCUSSION

6.1 Partial delegation and part identity

Partial delegation is a requirement for arbitrary compositions, where a composite may claim only some of a part's delegation surface. *Multi-rooted parts* result from this condition: a part's interfaces may therefore be rooted at two or more distinct identities, including the part itself and one or more composites.

This multi-rootedness makes a part's identity context-sensitive. References to `this`, whether implicit or explicit, resolve according to the context type of the reference. When that type is a claimed interface, ISD resolves `this` to the claiming composite. Otherwise, `this` remains the part itself.

The possession example in Section 3 demonstrates this directly. `Demon` claims only `Actor` from `Wizard`. Through delegation propagation, `Hero` consequently has separate roots for `Actor` and `Combatant`, making it a multi-rooted part.

6.2 Language Design Implications

Parts resolves the principal deficiencies of implementation inheritance:

- **Fragile base class** [2]: a composite's dependency surface is the interface contracts it explicitly claims, not a part's internal implementation; changes to its encapsulated internals do not propagate. Partial delegation completes the isolation: a composite is unaffected by changes to interfaces it did not claim.
- **Forced IS-A semantics**: reuse does not require a type relationship; the composite is a subtype of the interface, not of the part.
- **The *self* problem**: a hazard in forwarding-based composition where self-calls remain within the component rather than routing through the composite; resolved by ISD.
- **Diamond ambiguity**: explicit `@link(share=T.class)` resolution; unresolved diamonds are compile errors, not silent merges.
- **Encapsulation breach**: a composite may implement a subset of its parts' interfaces; those it does not implement are absent from its API by construction. `@internal` strengthens this further: it provides composite-scoped visibility, preventing designated interfaces from being claimed by any composite.
- **Abstract behavior**: abstract parts with deferred linkage (`asLink()`) preserve the template-method capability of abstract classes without coupling the hierarchy.

Implementation inheritance's power is open recursion. Its fragility is the surface that power ranges over: every overridable member. Parts keeps the open recursion but bounds the surface to the declared interface contract; self-calls still reach the composite's overrides, while the internals that make base classes fragile stay off it entirely.

The interface contract—not implementation types—is what makes dispatch constant-time: a fixed, statically-known interface set lets each interface's composite root be an array-indexed slot.

Restricting a part from exposing `this` as its concrete type is the static guarantee that makes the dispatch invariant hold.

Implementation inheritance and Parts are two bindings of the same open recursion: implementation inheritance binds it to the class hierarchy, where a subclass IS-A its superclass and inherits its entire implementation surface, while Parts binds it to a held object and a declared interface, where the composite HAS-A its parts and IS-A only their interfaces. That inheritance is open recursion, separable from subtyping, is Cook's [5, 6], and the reading of inheritance as composition is Bracha and Cook's [4]. Cook's model threads self as an explicit parameter; ISD keeps it implicit and per-interface, over independent runtime components with static interface contracts, at the cost of a virtual call.

6.3 Concurrency

A constructed composition is safe to share across threads. Each interface's `$selves` slot is written only while its final composite root (Invariant 1) is under construction; once that root is safely published the slot is never written again, so the JMM makes all wired state visible to any thread observing the root, with no synchronization on the dispatch path. The guarantee is thus scoped to a single composition, resting on the one-root invariant of Section 6. Parts may hold mutable state under the usual Java rules; dispatch adds no synchronization points.

6.4 Future Work

Per-interface composite roots as a general primitive. Assigning dispatch identity per interface rather than per object is a primitive whose uses may extend beyond this paper. Problems currently addressed through dynamic dispatch, AOP weaving, or collapsed component hierarchies may have cleaner solutions within it. The extent of that solution space is an open question.

Formal verification. A mechanized proof of Invariant 1 in a small-step operational semantics would provide assurance beyond the construction argument in Section 3. The model maps cleanly to a subset of Featherweight Java, a natural starting point.

Domain model and persistence integration. IS-A relationships in normalized relational schemas map naturally to the Parts model: each type owns its interface and its data independently, composed at construction with open recursion available across the composed object. Transactional systems are designed for this normalized, per-type form. Much of the pressure to denormalize comes from the object model: the table-per-hierarchy strategy—a common ORM default for IS-A—collapses the hierarchy into one table, the storage shadow of the single object implementation inheritance produces. Mapping such a schema to objects has meant a choice between an inheritance hierarchy that couples the types and hand-wired composition that gives up open recursion. Parts needs neither, keeping the types independent with open recursion intact, aligning the domain model with the normalized schema the relational model already favors. How that mapping is realized in a persistence framework is an open question.

Parts as a complete compositional alternative. Parts does not argue that implementation inheritance is wrong, but that it is no longer the only practical option. Any codebase written with implementation inheritance can in principle be expressed purely in Parts compositions; whether the result is better depends on the codebase. What the model offers is a compositional foundation that delivers open recursion without hierarchy and fosters low coupling and high cohesion through its interface contracts and independent parts. A systematic study of implementation inheritance patterns in real codebases would identify any remaining gaps and clarify where the compositional model offers the most advantage.

7 RELATED WORK

Static typing is the baseline requirement; models without it are included where relevant.

Forwarding. Kotlin's `by` keyword [25], Lombok's `@Delegate` [26], Scala 3's `export` keyword [24], and Go's struct embedding [27] forward to a supplied component. In all cases, self-calls resolve in the component's scope alone; they do not dispatch to the composite.

Scala traits / abstract type members. Odersky and Zenger [10] introduced statically-typed mixin composition with abstract type members. Scala's `with` clause linearizes traits into the target class at compile time, so self-calls dispatch through the outermost class in the linearized hierarchy. This gives open recursion, but it also fuses the component into the class: the trait's implementation is resolved at compile time, not supplied as an independent component at construction. A component implementation therefore cannot be chosen or replaced, and a single instance cannot be shared across composites. The fusion and the open recursion are inseparable: linearization is the single mechanism behind both, so in statically-typed languages fusing the component into the class became standard, and remains so in recent precisely-typed mixin systems [11]. Interface-scoped dispatch shows the fusion was never necessary: it obtains the same open recursion without abandoning independent components.

Mixins. Bracha and Cook [4] formalized mixins as parameterized abstract subclasses. Mixins extend through linearization: the mixin is applied to a chain at class definition time, not supplied as an independent component at construction. Self-calls dispatch through `this` in the linearized result.

Traits. Schärli et al. [7] proposed traits as stateless, composable units of behavior. Trait composition is a compile-time operation: methods are flattened into the target class, not supplied as independent components at construction. Self-calls dispatch through `this` in the composed class.

Typed self-recursion calculi. Chugh's ISOLATE [12] is a statically-typed calculus of functional objects that types a pattern of mixin composition with open recursion through self, without an explicit fixpoint. Its units of composition are *premethods*—functions defined apart from a host object and later mixed into it—so a method may be "defined independently of its host object," but the object itself is still assembled by fusion rather than supplied as an independent late-bound component. ISOLATE thus sits on the mixin side of the trade: open recursion without independent components, as in recent precisely-typed mixin systems [11].

Multiple inheritance. Stroustrup [13] introduced virtual base classes and virtual inheritance in C++ to handle diamonds. The result is a single linearized object with a merged vtable; components are not late-bound objects.

Dynamic single inheritance. Kniesel's DARWIN [14, 15] is a statically typed delegation-based model of single implementation inheritance: a base object is supplied at run-time through a `delegatee` field, its self-calls routing through the inheriting subtype. Being single inheritance, it admits at most one delegating (or forwarding) field per class [15, §6.5]; multiple delegation is *deliberately* excluded [15, §6.5.1], and a single receiver typed at one interface admits no multi-root composition [15, §5.5.4]. DARWIN thus achieves open recursion, but only over a single inherited base, not across arbitrary compositions; interface-scoped dispatch supplies both, by rooting identity in the interface rather than the object.

Deep delegation. DelphJ [16] is a research language in which a delegatee is supplied at run time and its self-calls route through the wrapping composite, giving open recursion over independent objects

under static typing. Two practical costs distinguish it from Parts. Forwarding is not automatic: a `subobject` "does not imply any automatic forwarding," so each composite requires manually written scripts using a "morphing" tool to identify the methods it forwards. And dispatch resolves over a runtime access path rather than a constant-time indexed slot. It is also difficult to determine how far the model extends to arbitrary, multi-root compositions (Section 6.1).

Family polymorphism. gbeta [20] (formalized by Ernst, Ostermann, and Cook [18]), CaesarJ [21], Object Teams/Java [22], and J& [19] target extensible class families, not independent components. Here, self-calls dispatch correctly within a family, but the object structure is defined statically by the family type, its members instantiated internally and not supplied independently: open recursion without independent components.

Delegation Layers. Ostermann's delegation layers [17] scale delegation from single objects to whole collaborations, composed at runtime, with operations routing to the composite collaboration rather than a single layer ("composition consistency"). But the composed unit is a collaboration, not an individual object, and its participants are nested classes redirected through the enclosing instance, not independently supplied: open recursion at collaboration granularity, without independent components.

Prototype-based delegation. Lieberman [3] introduced *open delegation*: an object may delegate unanswered messages to a prototype, with *self* remaining the original receiver throughout the chain. Self [8] realized this in a working language. These systems provide open recursion with late-bound components, but delegation is total and object-granular: a prototype supplies all behavior the delegating object does not, and cannot be scoped to a subset of interfaces, so neither partial delegation nor the multi-root dispatch it induces (§6.1) is expressible. Static type guarantees are absent by design. Cecil [9] is the statically-typed point in this space. A classless (prototype) language whose type system separates subtyping from code inheritance, it "does not incorporate dynamic inheritance, one of the most interesting features of Self," offering predicate objects as a "more structured but more restricted alternative" and attaching behavior only to statically-named objects rather than the run-time-supplied components at issue here.

Dynamic composition. Newspeak [23] makes all names late-bound and injects classes as first-class constructor arguments, enabling runtime reconfiguration and wiring of module objects. Its open recursion is that of mixin inheritance [4]: every class is a mixin applied to a superclass, fused into a single object; composed module objects are held by reference and reached by ordinary message send, so self-calls do not route across the composition. Static type checking is absent by design.

8 CONCLUSION

In static typing, the *self* problem with independent components is a large part of why object composition never displaced implementation inheritance: a self-call that reaches the composite's overrides is the one capability that made inheritance necessary rather than merely convenient. Parts resolves this by grounding identity in the interface rather than the object, recovering open recursion across independent components at the cost of a virtual call; the two properties are not inherently in conflict.

REFERENCES

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1994.

[2] L. Mikhajlov and E. Sekerinski. A study of the fragile base class problem. In *Proceedings of ECOOP*, LNCS 1445, pages 355–382, 1998.

[3] H. Lieberman. Using prototypical objects to implement shared behavior in object-oriented systems. In *Proceedings of OOPSLA*, pages 214–223, 1986.
[4] G. Bracha and W. Cook. Mixin-based inheritance. In *Proceedings of OOPSLA/ECOOP*, pages 303–311, 1990.
[5] W. Cook and J. Palsberg. A denotational semantics of inheritance and its correctness. In *Proceedings of OOPSLA*, pages 433–443, 1989.
[6] W. R. Cook, W. Hill, and P. S. Canning. Inheritance is not subtyping. In *Conference Record of POPL*, pages 125–135, 1990.
[7] N. Schärli, S. Ducasse, O. Nierstrasz, and A. Black. Traits: Composable units of behaviour. In *Proceedings of ECOOP*, LNCS 2743, pages 248–274, 2003.
[8] D. Ungar and R. B. Smith. Self: The power of simplicity. In *Proceedings of OOPSLA*, pages 227–242, 1987.
[9] C. Chambers. The Cecil language: Specification and rationale. Technical Report UW-CSE-93-03-05, University of Washington, 1993.
[10] M. Odersky and M. Zenger. Scalable component abstractions. In *Proceedings of OOPSLA*, pages 41–57, 2005.
[11] A. Fan and L. Parreaux. super-charging object-oriented programming through precise typing of open recursion. In *Proceedings of ECOOP*, LIPIcs 263, pages 11:1–11:28, 2023. DOI: 10.4230/LIPIcs.ECOOP.2023.11.
[12] R. Chugh. ISOLATE: A type system for self-recursion. In *Proceedings of ESOP*, LNCS 9032, pages 257–282, 2015. DOI: 10.1007/978-3-662-46669-8_11.
[13] B. Stroustrup. Multiple inheritance for C++. In *Proceedings of the EUUG Spring Conference*, 1987.
[14] G. Kniesel. Type-safe delegation for run-time component adaptation. In *Proceedings of ECOOP*, LNCS 1628, pages 351–366, 1999. DOI: 10.1007/3-540-48743-3_16.
[15] G. Kniesel. *Dynamic Object-Based Inheritance with Subtyping*. PhD thesis, University of Bonn, 2000.
[16] P. Gerakios, A. Biboudis, and Y. Smaragdakis. Forsaking inheritance: Supercharged delegation in DelphJ. In *Proceedings of OOPSLA*, pages 233–252, 2013.
[17] K. Ostermann. Dynamically composable collaborations with delegation layers. In *Proceedings of ECOOP*, LNCS 2374, pages 89–110, 2002.
[18] E. Ernst, K. Ostermann, and W. R. Cook. A virtual class calculus. In *Conference Record of POPL*, pages 270–282, 2006.
[19] N. Nystrom, X. Qi, and A. C. Myers. J&: Nested intersection for scalable software composition. In *Proceedings of OOPSLA*, pages 347–366, 2006.
[20] E. Ernst. Family polymorphism. In *Proceedings of ECOOP*, LNCS 2072, pages 303–326, 2001.
[21] I. Aracic, V. Gasiunas, M. Mezini, and K. Ostermann. An overview of CaesarJ. In *Transactions on Aspect-Oriented Software Development I*, LNCS 3880, pages 135–173, 2006.
[22] S. Herrmann. Object Teams: Improving modularity for crosscutting collaborations. In *Proceedings of NetObjectDays*, LNCS 2591, pages 248–264, 2002.
[23] G. Bracha, P. von der Ahé, V. Bykov, Y. Kashai, W. Maddox, and E. Miranda. Modules as objects in Newspeak. In *Proceedings of ECOOP*, LNCS 6183, pages 405–428, 2010.
[24] EPFL. Exports — Scala 3 reference. https://docs.scala-lang.org/scala3/reference/other-new-features/export.html, 2024.
[25] JetBrains. Delegation — Kotlin programming language. https://kotlinlang.org/docs/delegation.html, 2024.
[26] Project Lombok. `@Delegate` — Lombok features. https://projectlombok.org/features/Delegate, 2024.
[27] The Go Authors. Embedding — Effective Go. https://go.dev/doc/effective_go#embedding, 2024.
[28] A. Shipilev and contributors. JMH: Java Microbenchmark Harness. https://github.com/openjdk/jmh, 2013–2024.
[29] R. S. McKinney. manifold-parts. https://github.com/manifold-systems/manifold/tree/master/manifold-deps-parent/manifold-parts, 2026.